KUBERNETES CORE THEORY & ARCHITECTURE MANUAL

Systems Engineering Reference Handbook | Prepared By: Vikas Rawat | June 2026

1. The Container Orchestration Revolution

In modern cloud engineering, microservice architectures decouple heavy monolithic software stacks into small, isolated container processes. While engines like Docker handle the packaging and runtime lifecycle of individual localized containers, managing thousands of ephemeral application layers across hundreds of distributed physical nodes introduces immense operational complexity.

Manually configuring network subnets, tracking point-to-point application endpoints, managing structural failure recoveries, and provisioning high-availability horizontal scaling rules breaks operational efficiency.

Kubernetes (K8s) solves this systemic challenge. It serves as an open-source enterprise container orchestration ecosystem designed to automate deployment scaling, software resource scheduling, cross-cluster service routing, self-healing recoveries, and system storage state synchronization.

2. Cluster Topography: Control Plane vs. Worker Nodes

A production Kubernetes infrastructure is structurally bifurcated into a high-availability management layer (Control Plane) and an execution worker cluster grid layer (Worker Nodes).

2.1 The Control Plane (The Cluster Brain)

The Control Plane continuously observes cluster metrics, updates operational configurations, handles authentication validation parameters, and schedules core execution tasks:

- **API Server (`kube-apiserver`):** The centralized gateway front door of the cluster. It exposes the REST API layer, intercepting all administration queries via tools like `kubectl`, validating client permissions, and communicating with core cluster blocks.
- **etcd:** A distributed, consistent key-value datastore. It serves as the absolute single source of truth for the system, storing the state, hardware mappings, and telemetry rules of the environment.
- **Scheduler (`kube-scheduler`):** The resource matchmaker component. It monitors newly declared workload components and assigns them to specific physical nodes based on constraints like available memory/CPU resources, hardware affinity rules, and network requirements.
- **Controller Manager (`kube-controller-manager`):** The continuous self-healing orchestration core. It loops background processes to compare the cluster's *desired state* against its *actual running state*, adjusting components if a mismatch or machine failure occurs.

2.2 The Worker Nodes (Workload Execution Compute)

Worker nodes represent the physical virtual servers (e.g., AWS EC2 instances) allocated to execute your container payloads:

- **Kubelet:** The fundamental node agent interface. It ensures containers are running cleanly inside their assigned pods according to instructions received from the master API Server.

- **Kube-Proxy (`kube-proxy`):** The software-defined network switchboard running on each node. It maintains IP routing rules, configures virtual interface routing, and handles traffic balance distribution across pod targets.
- **Container Runtime:** The underlying container engine layer (such as `containerd`) responsible for pulling images from registries and running isolated application layers on host storage.

3. Core Kubernetes Objects & Abstract Entities

Engineers interact with Kubernetes by declaring logical abstractions called **Objects** using structured YAML code configurations.

OBJECT TYPE	SYSTEM ARCHITECTURE DEFINITION	OPERATIONAL IMPERATIVE
Pod	The smallest deployable unit of compute. It wraps one or more closely linked containers sharing a network space, loopback interface card, and storage.	Ephemeral unit; never deployed bare in enterprise production.
Deployment	A higher-level declarative controller wrapper that defines the desired state, replica counts, and rolling update strategies for a group of matching Pods.	Guarantees scale redundancy, automated self-healing, and rollback protection.
Service	An abstract network entry layer mapping a permanent internal IP and fixed DNS name to a dynamic group of shifting application Pods.	Provides microservice discovery and acts as an internal load balancer.

4. Advanced Infrastructure Management: Day 2 Theory

4.1 Layer 7 Application Traffic Routing: Ingress

While standard Kubernetes Services provide Layer 4 network routing, they are limited to basic IP and port configurations. To manage complex setups, engineers use an **Ingress Controller**. Ingress operates at Layer 7 (Application layer), managing external cluster HTTP/HTTPS access, enforcing SSL/TLS termination, and providing path-based or host-based routing (e.g., mapping `/api` to backend pods and `/static` to web components).

4.2 Decoupled Data Storage: PV and PVC Architecture

Because Pod lifecycles are highly transient, their local disk writes are volatile. To preserve stateful files, Kubernetes separates infrastructure storage into a two-part decoupled contract:

- **PersistentVolume (PV):** A cluster-wide storage block provisioned by an engineer or dynamically created via cloud integration plugins (such as an AWS EBS block). It persists independently of any individual Pod lifecycle.
- **PersistentVolumeClaim (PVC):** An isolation resource ticket created by a developer. It specifies requirements like capacity size (e.g., 50GB) and access criteria (e.g., `ReadWriteOnce`). Kubernetes interprets the PVC and binds it directly to a matching PV block to support stateful application operations.

4.3 Multi-Tenant Workspace Isolation: Namespaces

Namespaces provide virtual micro-cluster partitioning inside a single physical Kubernetes cluster. This helps segment multi-tenant enterprise environments into distinct boundaries (e.g., `dev`, `staging`, `prod`), preventing configuration conflicts and isolating resources across separate development teams.

> **Day 1 Interview Core Rule Summary:** Remember that the Control Plane components track the state of your infrastructure using the distributed `etcd` database. If an execution machine crashes, the `kube-controller-manager` triggers immediate remedial scheduling to bring the cluster back to its declared state without human intervention.